INFORMATICS INSTITUTE OF TECHNOLOGY

In Collaboration with

UNIVERSITY OF WESTMINSTER

Client-Server Architectures - Coursework

Name:Yadurshan Rajakumar

Module Code: 5COSC022C.2

IIT Student ID: 20220329

UoW Number: w1957961

Question 01: Project & Application Configuration

Explain the default lifecycle of a JAX-RS resource class. Is a new instance instantiated for every incoming request, or does the runtime treat it as a singleton? Elaborate on how this architectural decision impacts the way you manage and synchronize your in-memory data structures (maps/lists) to prevent data loss or race conditions.

Answer 01: Resource classes in JAX-RS follow a request based lifecycle by default, where a separate instance of the resource class is created for every HTTP request received, unlike using a singleton instance that would be used for all requests. This approach helps in processing each request individually and prevents interference between the data of different users in the resource class.

However in my implementation of the SmartCampusAPI, the data, such as rooms, sensors, and sensor readings, are stored in shared in memory data structures such as HashMaps that are found in the DataStore class. These data structures are declared as static, which means that data is shared across all requests and all users.

This allows the simultaneous use and modification of the same data by different requests. For example, in cases where two users try to create or delete rooms at the same time, this can lead to problems like data inconsistency.

The resource classes, though inherently safe because of their lifecycle that involves only requests, must be managed properly. In this implementation, no synchronization methods are explicitly implemented because it is memory-based and relatively simple. However, a real

world production level implementation will involve using thread-safe constructs like ConcurrentHashMap and/or the use of databases in order to prevent data corruption.

Question 02: The ”Discovery” Endpoint

Why is the provision of ”Hypermedia” (links and navigation within responses) considered a hallmark of advanced RESTful design (HATEOAS)? How does this approach benefit client developers compared to static documentation?

Answer 02: The concept of Hypermedia, known as “HATEOAS” (Hypermedia as the Engine of Application State), is considered a key attribute of advanced RESTful architecture. The inclusion of links in the responses of the server allows it to guide the client through the API dynamically. The client is not required to know all endpoints; instead, the API provides details on how actions can be performed.

The advantage for developers is that there is no need to rely on documentation and URLs since any modifications made to the API structure will still ensure that clients continue functioning appropriately by adhering to the links provided in the response.

In my implementation of SmartCampusAPI, a basic form of hypermedia is implemented through the discovery endpoint of ‘api/v1’. This endpoint returns a structured output model containing information about the API such as the version number, administrative contact details, and available endpoints such ‘api/v1/rooms’ or ‘api/v1/sensors’. This approach allows the clients to understand about the API and the endpoints available before engaging with the API.

Question 03: Room Resource Implementation

When returning a list of rooms, what are the implications of returning only IDs versus returning the full room objects? Consider network bandwidth and client side processing.

Answer 03: There are two main approaches to returning a list of rooms, returning only the IDs of the rooms or returning the full objects of the rooms consisting of all the details. Each approach has its own implications on the network bandwidth and client side processing.

Returning only the IDs will result in small responses that make the process more efficient and less bandwidth-consuming. This approach is beneficial when dealing with multiple rooms since it takes less time to transfer. However, the client will need to make several requests in order to get all the information about each room.

On the contrary, when sending full room objects, all necessary information is sent in a single response. This approach enables the client to utilize the data without having to send additional requests, simplifying the process of development. However, this approach increases the response size, which can be problematic when the database becomes large.

In my implementation of SmartCampusAPI, the API returns full objects for rooms, including fields such as ID, name, capacity, and sensors. This design approach increases the ease-of-use of the system for the client because everything is presented in a single response. This is suitable for my implementation since the amount of data is small.

Question 04: Room Deletion & Safety Logic

Is the DELETE operation idempotent in your implementation? Provide a detailed justification by describing what happens if a client mistakenly sends the exact same DELETE request for a room multiple times.

Answer 04: Yes, the DELETE operation in this implementation is idempotent. In my implementation of SmartCampusAPI, when a client sends a DELETE request to remove a room, if the room already exists and has no sensors attached to it, then that room gets deleted from the DataStore. The first delete operation works fine and succeeds, returning a successful response.

If a similar DELETE request is made for the exact room later, there won't be a detail of the room left in the system anymore. Thus, the API will return a "Room not found" message, which will have a status code of 404. However, this does not change the state of the system, because the room was already deleted.

This demonstrates idempotency because repeated DELETE queries does not cause any further changes beyond the first DELETE action. The system is always in the same state regardless of how many times the query is executed.

Question 05: Sensor Resource & Integrity

Explain the technical consequences if a client attempts to send data in a different format, such as text/plain or application/xml. How does JAX-RS handle this mismatch?

Answer 05: In my implementation of SmartCampus API, the @Consumes (MediaType.APPLICATION_JSON) is the annotation used to define that the service API will receive data in JSON form for POST requests, implying that JSON will be the form of incoming request bodies.

When the client uses an alternative media type in sending the data, the JAX-RS will not be able to handle the request since the media type is not compatible with the required media type. The client will therefore receive a 415 Unsupported Media Type, which means the server received the request but was unable to process it because of the wrong format used.

This behaviour ensures that only well-formatted data is received by the API, maintaining consistency and avoiding any parsing errors. In the context of this project, for example, any POST call intended to create either a room or a sensor must be made using JSON formatted data; otherwise, the call will not reach the resource method.

Question 06: Filtered Retrieval & Search

You implemented this filtering using @QueryParam. Contrast this with an alternative design where the type is part of the URL path (/api/vl/sensors/type/CO2). Why is the query parameter approach generally considered superior for filtering and searching collections?

Answer 06: In my implementation of SmartCampus API, the filtering of sensors by type is implemented using @QueryParam, /api/v1/sensors?type=CO2. This approach is often used when there is a need to filter/search elements in a collection. An alternative approach

involves including the type in the path of the URL, /api/v1/sensors/type/CO2. However, this solution does not provide enough flexibility when it comes to filtering.

Query parameters are more suitable for the filtering operation since they represent optional criteria instead of immutable resource identities. Query parameters are used for filtering a set of resources, whereas path parameters are used for pointing to one particular resource identity.

The adoption of query parameters will allow us to implement multiple filtering operations easily. For example including other filter parameters like status and ranges in the URL path /api/v1/sensors?type=CO2&status=ACTIVE.

In this implementation, using @QueryParam makes the API more organized and understandable for users since we distinguish between resource identity and filtering. It also complies with REST principles, as we perform filtering of a resource set instead of creating an extra resource URL path.

Question 07: The Sub-Resource Locator Pattern

Discuss the architectural benefits of the Sub-Resource Locator pattern. How does delegating logic to separate classes help manage complexity in large APIs compared to defining every nested path (e.g., sensors/{id}/readings/{rid}) in one massive controller class?

Answer 07: The sub-resource locator pattern refers to a technique used in JAX-RS that uses different classes to manage nested resources as opposed to combining all functionalities

within one class for managing nested resources. This helps with organizing the API because responsibilities are assigned to appropriate classes.

The key benefit that comes with such a design technique is improved code structure. It is no longer necessary to have one big controller file that deals with all the endpoints, for rooms, sensors, and sensor data. Instead, it is possible to handle these resources separately, which results in better code readability and ease of maintenance.

This approach simplifies large APIs by making them less complex. If all the nested paths were implemented inside one class, then the resulting code would have been hard to debug and modify as well. Assigning the responsibility of doing something related to reading from another class makes it easier to adhere to the rule of having only one responsibility per class.

Another benefit in using this approach is scalability. Since there is minimum interference between classes while developing features, therefore adding new features becomes easy and does not introduce defects into the system. In my implementation, sensor readings have been implemented as a sub-resource of sensors which follows this methodology and improves the modularity of our API.

Question 08: Dependency Validation (422 Unprocessable Entity)

Why is HTTP 422 often considered more semantically accurate than a standard 404 when the issue is a missing reference inside a valid JSON payload?

Answer 08: HTTP 422 (Unprocessable Entity) is considered semantically more suitable for use when the issue arises out of the absence of the reference inside a correctly structured JSON content, because even if the HTTP request itself is correct, its contents are not.

The 404 error is usually received when the end-point that has been queried does not exist, for example, querying a non-existing URL on the server. In the current scenario, the endpoint does actually exist and the request itself is properly formatted.

In my implementation of SmartCampusAPI, upon receiving a request by the client for the creation of a sensor with an invalid roomID, the JSON format used in the payload of the request is correctly structured, but there is a logical mistake in the data sent.

Thus, returning a 422 HTTP response status is the most suitable action to take since it clearly shows that the request was understood by the server but could not be processed because of problems with the semantics of the request.

Question 09: The Global Safety Net

From a cybersecurity standpoint, explain the risks associated with exposing internal Java stack traces to external API consumers. What specific information could an attacker gather from such a trace?

Answer 09: From a cybersecurity perspective, exposing the internal Java stack traces to external API users poses risks since it exposes important information about the inner workings of the system and how it functions.

The stack trace may contain information such as the names of the classes being used, packages, methods used, file paths, and the actual implementation of the logic. Using this information, an attacker may be able to decipher how the system works and where its vulnerabilities lie. For example, knowing the exact name of certain classes or library files used makes it easier to target certain vulnerabilities.

Stack traces may also expose the implementation information that an attacker could use, including database connections, frameworks used, or even errors. Attackers would be able to use this information to create more efficient attacks on the system.

In my implementation of SmartCampus API, this risk is reduced using a GlobalExceptionMapper that captures unexpected errors and sends a message stating that there was an internal error within the server instead of exposing detailed stack traces of the API. This ensures that sensitive internal information is not leaked to the clients.

Question 10: API Request & Response Logging Filters

Why is it advantageous to use JAX-RS filters for cross-cutting concerns like logging, rather than manually inserting Logger.info() statements inside every single resource method?

Answer 10: Utilizing the JAX-RS filter in order to tackle issues related to cross-cutting concerns like logging has a number of benefits due to the fact that the logging process does not affect the core business logic.

Using logging calls (Logger.info()) within every resource method will result in repetition and make the code difficult to maintain. Any change made in the area of logging will involve changing the code in many methods.

In my implementation of SmartCampusAPI, LoggingFilter has been utilized in this project to automatically log all requests and responses. This means that logging will be abstracted from classes such as RoomResource or SensorResource and handled in a single instance.

This technique is an implementation of the separation of concerns design pattern. Every aspect of software will have only one responsibility in the project, which means future additions, for instance authentication or monitoring, will be easy to incorporate.